Locators and Element Identification

COURSE SPECIFICATIONS

Master resilient element identification with user-facing locators, strict mode resolution, CSS & XPath trade-offs, web-first assertions, Playwright Inspector debugging, and AI-assisted locator engineering.

- Track: Playwright Automation with JavaScript + AI • Level: Beginner to Intermediate
- Focus: Locators, Strict Mode, Web-First Assertions • Prerequisites: Day 1–3 Concepts
- Tools: Playwright Inspector (--debug), Codegen, AI • Test Runner: @playwright/test (.js)

Day 4 Learning Objectives

- Explain what a locator is in Playwright and how lazy evaluation and auto-waiting differ from traditional DOM element handles.
- Identify elements using Playwright user-facing locators (getByRole, getByLabel, getByPlaceholder, getByText, getByTestId).
- Understand when and how to fall back to CSS selectors and XPath, and recognize why absolute XPath is an anti-pattern.
- Handle multiple matching elements and strictness violations using locator chaining and .filter({ hasText }).
- Write resilient web-first assertions (toBeVisible, toHaveText, toBeEnabled, toHaveCount) that retry automatically.
- Debug locator issues using the Playwright Inspector (npx playwright test --debug), page.pause(), and codegen.
- Leverage AI tools to generate, evaluate, and refine robust locators from raw HTML snippets without introducing fragile dependencies.

In browser automation, element identification is the single most critical factor determining whether your test suite is rock-solid or plagued by flakiness.

What is a Locator in Playwright?

A **Locator in Playwright is not a static reference to a DOM node in the browser's memory. Instead, a Locator is a **recipe or view** that describes *how* to find an element on the page at any given moment.**

```javascript
// This does NOT search the DOM immediately:
const submitButton = page.getByRole('button', { name: 'Submit Application' });

// The DOM search happens right here when the action or assertion is called:
await submitButton.click();
```

Locators vs Traditional Element Handles (Selenium / Puppeteer)

In legacy tools like Selenium or Puppeteer, finding an element was an eager operation:

1. In Selenium: `const el = driver.findElement(By.id("btn"));` immediately queried the browser DOM.

2. If the page was still loading or re-rendering a React component, Selenium immediately threw `NoSuchElementException` or later `StaleElementReferenceException`.

3. In Playwright, locators are **lazy. Every time you perform an action (like `.click()` or `.fill()`), Playwright recalculates the locator, locates the element afresh, auto-waits for it to be visible, enabled, and stable, and executes the action safely.**

Why Positional Selectors Break

Many beginners copy selectors like `div > div:nth-child(3) > button` or full XPath `/html/body/div[1]/section/div[2]/form/div[3]/button`.

• These positional selectors are tightly coupled to the exact structural layout of the HTML tree.

• If a developer adds a notification banner, wraps an input in a new `<div>`, or changes the order of columns, your test immediately breaks even though the user functionality works fine!

• Resilient locators target what the user sees and interacts with, not the arbitrary internal hierarchy of CSS tags.

Comparing Locator Resilience

Strategy	Example	Flakiness Risk	Recommendation
User-Facing Accessible Locator	`page.getByRole('button', { name: 'Log in' })`	Extremely Low	**Gold Standard** (matches user perception)
Visible Form Label	`page.getByLabel('Email Address')`	Very Low	**Best for inputs and forms**
Semantic Test ID	`page.getByTestId('checkout-pay-btn')`	Low	Use when no unique visible label exists
Scoped CSS Selector	`page.locator('form.login-card input[type="email"]')`	Medium	Acceptable for complex custom components
Positional CSS / Full XPath	`/html/body/div[2]/div/div[4]/button`	Critical	**Anti-Pattern** (Breaks on minor redesigns)

Lazy Evaluation in Action (You can declare locators at the top of a test or in a Page Object constructor. Playwright will not query the DOM until you perform an action like await loginBtn.click(). If the page navigates or re-renders, the locator seamlessly resolves to the fresh element.)

```
// tests/example.spec.js
const { test, expect } = require('@playwright/test');

test('demonstrates lazy locator evaluation', async ({ page }) => {
  await page.goto('https://practice.expandtesting.com/login');

  // Declared lazily - no DOM query happens here:
  const usernameInput = page.getByLabel('Username');
  const passwordInput = page.getByLabel('Password');
  const loginButton = page.getByRole('button', { name: 'Login' });

  // DOM queries happen during action execution with auto-waiting:
  await usernameInput.fill('practice');
  await passwordInput.fill('SuperSecretPassword!');
  await loginButton.click();

  await expect(page.getByRole('alert')).toContainText('You logged into a secure area!');
});
```

Playwright prioritizes user-facing locators that reflect how actual users and assistive technologies perceive the webpage.

1. page.getByRole(role, { name })
This is the recommended first-choice locator for almost every interactive element:
• **Common roles: `button`, `link`, `heading`, `textbox`, `checkbox`, `radio`, `combobox`, `dialog`, `tab`.**
• **The name option: Refers to the accessible name (the visible text, `aria-label`, or linked label).**

```javascript
// Matches <button>Save Changes</button>
await page.getByRole('button', { name: 'Save Changes' }).click();

// Matches <h2>Account Settings</h2> with level matching
await expect(page.getByRole('heading', { name: 'Account Settings', level: 2 })).toBeVisible();

// Matches <input type="checkbox"> Agree to terms
await page.getByRole('checkbox', { name: 'Agree to terms' }).check();
```

2. page.getByLabel(text)
The ideal locator for form inputs associated with a `<label>` tag or `aria-labelledby`:

```html
<label for="user-email">Email Address</label>
<input id="user-email" type="email" />
```

```javascript
await page.getByLabel('Email Address').fill('alex.student@rootaccess.org');
```

3. page.getByPlaceholder(text)
Use this when inputs do not have an explicit `<label>` tag, but have placeholder text:

```html
<input type="text" placeholder="Search courses or topics..." />
```

```javascript
await page.getByPlaceholder('Search courses or topics...').fill('Playwright');
```

4. page.getByText(text, { exact })
Best for non-interactive elements like paragraphs, status badges, banners, and confirmation text:

```javascript
await expect(page.getByText('Order #10492 confirmed!')).toBeVisible();
// Exact substring match:
await expect(page.getByText('Pending Approval', { exact: true })).toBeVisible();
```

5. page.getByTestId(id)
When an element lacks accessible text or semantic labels (such as an empty SVG icon button or complex canvas container), ask developers to add a `data-testid` attribute:

```html
```

<button data-testid="cart-drawer-trigger"><svg .../></button>
```
```javascript
**await page.getByTestId('cart-drawer-trigger').click();**
```

The Playwright Locator Hierarchy Pyramid
```
        %²
       / \    1. getByRole() - Buttons, links, headings, checkboxes
      /   \   2. getByLabel() - Form input fields
     /     \  3. getByPlaceholder() - Unlabeled search/input fields
    /       \ 4. getByText() - Status badges, labels, alerts
   /         \5. getByTestId() - Icon buttons & dynamic widgets
  /___________\6. CSS / XPath - Fallback only for legacy markup
```

Complete Form Interaction with User-Facing Locators (A clean registration script using exclusively user-facing locators in standard JavaScript.)

```
// tests/registration.spec.js
const { test, expect } = require('@playwright/test');

test('registers new user with accessible locators', async ({ page }) => {
  await page.goto('https://practice.expandtesting.com/register');

  await page.getByRole('heading', { name: 'Create Account', level: 1 }).waitFor();
  await page.getByLabel('Full Name').fill('Alex Student');
  await page.getByLabel('Email Address').fill('alex@rootaccess.org');
  await page.getByLabel('Password').fill('StrongPassw0rd!');
  await page.getByRole('checkbox', { name: 'I agree to the Terms of Service' }).check();
  await page.getByRole('button', { name: 'Register Now' }).click();

  await expect(page.getByRole('alert')).toContainText('Account created successfully');
});
```

While user-facing locators are preferred, senior QA engineers must know when and how to safely use CSS selectors and XPath for legacy codebases or complex web widgets.

CSS Selectors: Syntax and Best Practices
CSS selectors identify elements using HTML tag names, IDs, classes, and attributes:
• **ID selector: `page.locator('#user-profile')` — Fast, but only reliable if the ID is static and unique (not dynamically generated like `#input-19283`).**
• **Attribute selector: `page.locator('input[name="postal_code"]')` — Great when name attributes are fixed.**
• **Scoped selector: `page.locator('form#checkout-form input[type="text"]')` — Targets inputs inside a specific form.**

Playwright-Enhanced CSS Selectors
Playwright extends standard CSS with powerful pseudo-classes:
• `:has-text("...")` — `page.locator('.course-card:has-text("Playwright")')`
• `:visible` — `page.locator('.dropdown-menu:visible')` (only matches currently rendered, non-hidden elements).

XPath: The Good, the Bad, and the Dangerous
XPath (XML Path Language) navigates through elements and attributes in an XML/HTML document:
• **Relative XPath (Acceptable when necessary):
`page.locator('//div[@data-sku="PLAYWRIGHT-01"]//button')`
• **Absolute XPath (NEVER USE):
`page.locator('/html/body/div[1]/div[2]/main/div/section[2]/div/article[3]/div/button')`
***Why absolute XPath is catastrophic:* Any tiny DOM change (adding a banner, wrapping a div, changing margins) completely breaks the test path.**

CSS vs XPath vs User-Facing Locators Comparison

Feature	User-Facing Locators	CSS Selectors	XPath
Readability	Highest (`getByRole('button')`)	Good (`button.primary`)	Low (`//button[contains(@class...)]`)
Resilience to Restyling	Immune to CSS class renames	Breaks on class renames	Highly fragile if positional
Accessibility Verification	Enforces a11y standards	Ignores accessibility	Ignores accessibility
Execution Performance	Fast native CDP traversal	Fast native browser engine	Slightly slower XML evaluation
Best Used For	90% of all UI tests	Custom styling, inputs by name	Deep sibling traversal in legacy tables
```

Comparing Fragile XPath to Scoped Attribute Selector (Refactoring a brittle legacy XPath selector into a clean, resilient attribute locator.)

```
// 'L Brittle Absolute XPath:
await page.locator('/html/body/div[2]/section/form/div[2]/input').fill('admin');

// 'L Fragile class-dependent CSS:
await page.locator('div.col-md-6 > div.form-group > input.form-control-lg').fill('admin');

// '  Resilient User-Facing or Attribute Locator:
await page.getByLabel('Username').fill('admin');
// Or if label is missing:
await page.locator('input[name="username"]').fill('admin');
```

One of the most important concepts in modern Playwright is **Strict Mode**.

What is Playwright's Strict Mode?
When you instruct Playwright to perform an action on a locator (like `.click()` or `.fill()`), Playwright requires that the locator resolves to **exactly one single element.**
• If 0 elements match: Playwright auto-waits until the timeout and throws a timeout error.
• If **2 or more elements match: Playwright immediately throws a **Strictness Violation Error**:**
```
**Error: locator.click: Error: strict mode violation: getByRole('button', { name: 'Add to Cart' }) resolved to 5 elements:**
**  1) <button class="btn">Add to Cart</button> aka getByRole('button', { name: 'Add to Cart' }).first()**
**  2) <button class="btn">Add to Cart</button>**
**  3) <button class="btn">Add to Cart</button>**
**  ...**
```

Why Strict Mode is a Superpower
In legacy tools like Selenium or Cypress, calling `click()` when 5 elements match might click the first one invisibly. If the page order changed, your test clicked the wrong item without failing, leading to mysterious bugs and silent test corruption. Playwright forces you to be explicit!

How to Narrow Down Locators

1. Locator Filtering with .filter({ hasText })
Filter a collection of container elements down to the specific one containing unique text:
```javascript
// Find all product cards, filter by the specific course title, then find the button inside it:
const playwrightCard = page
  .locator('.product-card')
  .filter({ hasText: 'Playwright Automation with JavaScript' });

await playwrightCard.getByRole('button', { name: 'Enroll Now' }).click();
```

2. Locator Filtering with Child Elements (.filter({ has }))
Filter containers that contain a specific child locator:
```javascript
// Find table rows that contain an active status badge:
const activeRow = page
  .getByRole('row')
  .filter({ has: page.getByText('Active', { exact: true }) });

await activeRow.getByRole('button', { name: 'Edit' }).click();
```

3. Locator Chaining
Chain locators from parent to child:
```javascript
const checkoutModal = page.getByRole('dialog', { name: 'Review Order' });
await checkoutModal.getByRole('button', { name: 'Confirm Payment' }).click();
```

4. When to use .first(), .last(), and .nth()
• `locator.first()` / `locator.last()` / `locator.nth(index)` explicitly pick by position.
• **Caution:** Use positional indexers sparingly. If products or list items reorder dynamically, your test might target the wrong item. Always prefer text filtering where possible!

5. Iterating and Counting Elements
```javascript
const items = page.getByRole('listitem');
const count = await items.count();
console.log(`Found ${count} items in the cart`);

// Iterate through elements:
for (const item of await items.all()) {
  console.log(await item.textContent());
}
```

Targeting a Specific Row in a Dynamic Table (Using filter and chaining to click Delete on a specific user without relying on row indices.)

```javascript
// tests/users-table.spec.js
const { test, expect } = require('@playwright/test');

test('deletes user by email without positional index', async ({ page }) => {
  await page.goto('https://practice.expandtesting.com/tables');

  // Filter table rows to the one containing the user's email:
  const userRow = page.getByRole('row').filter({ hasText: 'fbach@yahoo.com' });

  // Within that specific row, click the Delete action:
  await userRow.getByRole('link', { name: 'delete' }).click();

  // Verify the row is no longer visible:
  await expect(userRow).toBeHidden();
});
```

Assertions are the verdict of your automated test. In Playwright, you should always use **Web-First Assertions**.

Web-First Assertions vs Generic Assertions
Consider these two ways to verify an element is visible:
```javascript
// 'L Generic Assertion (Anti-Pattern - DO NOT USE):
const isVisible = await page.getByRole('button').isVisible();
expect(isVisible).toBe(true);

// '  Web-First Assertion (Gold Standard):
await expect(page.getByRole('button')).toBeVisible();
```

Why Generic Assertions Cause Flaky Tests
• When you run `await page.getByRole('button').isVisible()`, Playwright checks the element once at that microsecond. If an animation is fading in or an API call is finishing, `isVisible()` returns `false` immediately, and `expect(false).toBe(true)` fails the test!
• When you run `await expect(locator).toBeVisible()`, Playwright enters a smart retry loop. It polls the DOM every 100 milliseconds for up to 5 seconds until the condition becomes true. If it becomes visible after 350ms, your test passes seamlessly without arbitrary sleep delays!

Core Playwright Web-First Assertions

Assertion	Purpose	Example
`toBeVisible()`	Verifies element is rendered and visible	`await expect(banner).toBeVisible();`
`toBeHidden()`	Verifies element is detached or hidden	`await expect(loadingSpinner).toBeHidden();`
`toHaveText()`	Matches full or trimmed text	`await expect(header).toHaveText('Welcome Alex');`
`toContainText()`	Matches substring text	`await expect(alert).toContainText('Success');`
`toHaveValue()`	Checks form input value	`await expect(emailInput).toHaveValue('alex@test.com');`
`toBeEnabled()`	Confirms button/input is clickable	`await expect(submitBtn).toBeEnabled();`
`toBeDisabled()`	Confirms button is grayed out/disabled	`await expect(submitBtn).toBeDisabled();`
`toHaveCount()`	Verifies exact number of matching nodes	`await expect(productCards).toHaveCount(6);`
`toHaveAttribute()`	Checks HTML attribute value	`await expect(link).toHaveAttribute('href', '/login');`

Customizing Assertion Timeouts
By default, web-first assertions poll for 5000ms (5 seconds). You can adjust timeouts per assertion:
```javascript
// Allow up to 10 seconds for a slow third-party payment iframe:

await expect(page.getByText('Payment Succeeded')).toBeVisible({ timeout: 10000 });
```

**Testing a Search Filter with Web-First Assertions (Using toHaveCount and toContainText to verify live search filtering.)**

```
// tests/search.spec.js
const { test, expect } = require('@playwright/test');

test('filters search results dynamically', async ({ page }) => {
  await page.goto('https://practice.expandtesting.com/books');

  const cards = page.locator('.book-card');
  // Initially verify 8 books are displayed:
  await expect(cards).toHaveCount(8);

  // Type into search filter:
  await page.getByPlaceholder('Search books').fill('JavaScript');

  // Web-first assertion auto-waits until list filters down:
  await expect(cards).toHaveCount(2);
  await expect(cards.first()).toContainText('JavaScript');
});
```

When a locator fails or matches multiple elements, Playwright provides first-class debugging tools to inspect the live DOM without guessing.

1. Playwright Inspector (--debug)
Run your test in debug mode to open the interactive Playwright Inspector alongside a live browser:
```bash
npx playwright test tests/day4-locators.spec.js --debug
```
• **Step Over: Advance through your test line by line.**
• **Pick Locator Tool: Click the crosshair icon in the Inspector, hover over any element in the browser, and Playwright immediately generates the optimal user-facing locator for you!**
• **Live Locator Evaluation: Type any locator into the Inspector search box to test how many elements it matches in real time.**

2. Programmatic Pausing with page.pause()
Place `await page.pause();` directly in your JavaScript test code where an issue occurs:
```javascript
test('diagnosing tricky locator', async ({ page }) => {
** await page.goto('https://practice.expandtesting.com');**
** // Script pauses here and opens Inspector:**
** await page.pause();**
** await page.getByRole('button', { name: 'Subscribe' }).click();**
});
```

3. Code Generator (npx playwright codegen)
Playwright can record your actions and generate JavaScript code on the fly:
```bash
npx playwright codegen https://practice.expandtesting.com
```
• **Golden Rule for Senior QA: Codegen is an outstanding discovery tool, but do **not** accept generated code blindly. Review generated selectors, remove redundant clicks, and convert generic locators to accessible semantic roles if needed.**

4. Reading Locator Error Messages
When an assertion or action fails, Playwright gives you crystal-clear diagnostic information:
1. **Timeout Error:**
** `locator.click: Timeout 5000ms exceeded. waiting for getByRole('button', { name: 'Save' })`**
** *Diagnosis:* Element never appeared in the DOM, or remained `display: none` / obscured.**
2. **Strictness Violation:**
** `resolved to 3 elements`**
** *Diagnosis:* Locator is too broad; use `.filter({ hasText })` or locator chaining.**
```

Quick Debugging with console.log and count() (Print locator counts in your test during development to debug element resolution.)

```
const cards = page.locator('.course-item');
const count = await cards.count();
console.log('Detected course items:', count);

if (count > 1) {
  const targetCard = cards.filter({ hasText: 'Automation' });
  await targetCard.getByRole('button').click();
}
```

Generative AI tools (ChatGPT, Claude, GitHub Copilot) can accelerate test authoring, but naive AI prompts often generate brittle XPath or volatile CSS classes. Senior automation engineers know how to prompt AI effectively and rigorously verify suggestions.

How to Write Effective AI Prompts for Locators
When asking an LLM to generate or fix locators, follow this 4-step prompt pattern:
1. **Provide the Exact HTML Snippet: Copy the relevant DOM node and its parent container.**
2. **State Target Action & Framework: Specify Playwright Test with JavaScript (.js).**
3. **Set Architecture Constraints: Explicitly instruct the AI: *"Prefer getByRole, getByLabel, or getByPlaceholder. Avoid absolute XPath, array indices, and transient utility CSS classes."***
4. **Request Explanation & Resilience Rating: Ask the AI to justify why the locator won't break if classes or layout change.**

Proven AI Locator Prompt Template
```
**I am writing an automated test in Playwright using standard JavaScript (.js).**
**Here is the HTML markup for the component:**
**[PASTE HTML SNIPPET HERE]**
Task:
1. Provide the most resilient Playwright locator to interact with [SPECIFY ELEMENT, e.g. "the Add to Cart button for the Wireless Headphones"].
2. Follow Playwright best practices: prefer user-facing locators (getByRole, getByLabel, getByText) or .filter({ hasText }).
3. Strictly avoid absolute XPath, positional indices (.first()/.nth()), or dynamic CSS utility classes.
4. Provide the exact assertion using web-first expect(locator).toBeVisible().
```

Evaluating and Refining AI Suggestions
Always evaluate AI-generated locators against the **4-Question Checklist:**
1. *Is it user-facing?* Does it match what the human sees or accessibility roles?
2. *Is it unique?* Will it violate strict mode if 3 other items appear?
3. *Is it decoupled from design?* Will changing Tailwind classes (`bg-blue-600 px-4`) break it?
4. *Did you verify it in the real DOM?* Always test it in Playwright Inspector or terminal before committing!

Reviewing and Upgrading an AI-Generated Locator (How a senior engineer transforms a naive AI suggestion into production-grade code.)

```
// 'L Naive AI Suggestion:
await page.locator('//div[contains(@class,"card")][2]//button').click();

// Ø=Ý  Engineer Review:
// Problem: Relies on XPath and [2] positional index. If items re-order, wrong item is clicked!

// '  Refined Production-Grade Locator:
const productCard = page.locator('.product-card').filter({ hasText: 'Professional Headset' });
await productCard.getByRole('button', { name: 'Add to Cart' }).click();
```

(getByRole, getByPlaceholder), handle strict mode violations using .filter({ hasText }) or locator chaining, write at least 3 web-first assertions, document an AI prompt & review workflow, and verify execution in the live DOM.

Task: Submit your tests/day4-locators.spec.js code, terminal execution log, and written explanation meeting all 10 rubric criteria.

Assignment Starter Code (tests/day4-locators.spec.js)

```
// tests/day4-locators.spec.js
// Playwright Automation with JavaScript + AI — Day 4
const { test, expect } = require('@playwright/test');

test.describe('E-Commerce Product Catalog — Locator Mastery', () => {
  test('accurately locates elements using resilient user-facing strategies', async ({ page }) => {
    // 1. Navigate to target application
    await page.goto('https://practice.expandtesting.com/books');

    // 2. Target Element 1: Accessible heading via getByRole
    // Strategy: getByRole('heading', { level: 1 }) is resilient to DOM reorganization and styling changes
    const pageHeader = page.getByRole('heading', { name: 'Book Store', level: 1 });
    await expect(pageHeader).toBeVisible();

    // 3. Target Element 2: Search input via getByPlaceholder or getByLabel
    // Strategy: getByPlaceholder provides clear human-facing intent without relying on dynamic input IDs
    const searchInput = page.getByPlaceholder('Type to search...');
    await searchInput.fill('Playwright');
    await expect(searchInput).toHaveValue('Playwright');

    // 4. Target Element 3 & Narrowing Multiple Matches: Filter specific product card
```

10 Rubric Requirements for Day 4 Assignment:

1. Identifies and targets at least 3 distinct elements (headings, inputs, buttons, cards)
2. Uses Playwright user-facing accessible locators (getByRole, getByLabel, getByPlaceholder)
3. Contains at least 3 auto-retrying web-first assertions (toBeVisible, toHaveText, toBeEnabled, toHaveCount)
4. Demonstrates resolving multiple elements using .filter({ hasText }) or locator chaining
5. Documents AI prompt and locator review/refinement workflow
6. Includes realistic terminal test execution output (e.g. 1 passed)
7. Clear explanation of locator selection rationale in written notes or code comments
8. Addressed strict mode disambiguation without fragile positional indices (.first() / .nth())
9. Explains real DOM verification methodology using Playwright Inspector (--debug) or locator.count()
10. Syntactically valid JavaScript test structure using test, expect, and async ({ page })

Q1. What is a Locator in Playwright, and how does it differ from a DOM element handle in Selenium?

[Difficulty: Beginner] What Interviewer Evaluates: Assesses understanding of lazy evaluation, auto-waiting, and how Playwright avoids stale element errors.

Suggested Answer: In Playwright, a Locator is a lazy description or view of how to find elements on the page. Unlike Selenium WebElement handles, which query the DOM eagerly and throw StaleElementReferenceException when the DOM re-renders, a Playwright Locator re-queries the DOM right before every action or assertion and auto-waits for actionability checks (visible, stable, enabled).

Follow-Up Question: "What happens under the hood when you call await page.getByRole("button").click()?"

Q2. Why does Playwright recommend user-facing locators like getByRole and getByLabel over CSS classes or XPath?

[Difficulty: Beginner] What Interviewer Evaluates: Tests appreciation for test resilience, accessibility standards, and maintainability.

Suggested Answer: User-facing locators emulate how human users and assistive technologies perceive the webpage. They verify that the element has the correct accessibility role and accessible text. Furthermore, CSS classes and DOM hierarchies frequently change during frontend redesigns, whereas user-facing roles and labels remain stable.

Follow-Up Question: "In what rare situations would you fall back to a data-testid or CSS selector?"

Q3. What is Playwright Strict Mode, and what happens when a locator matches multiple elements during an action?

[Difficulty: Intermediate] What Interviewer Evaluates: Evaluates understanding of strict mode violations and how to resolve them safely.

Suggested Answer: Playwright Strict Mode ensures that any locator passed to an action (like click or fill) matches exactly one DOM element. If more than one element matches, Playwright immediately throws a Strict Mode Violation Error. This prevents tests from accidentally interacting with the wrong element when lists or layouts change.

Follow-Up Question: "Why is adding .first() to bypass a strict mode error considered a bad practice in regression testing?"

Q4. How do you target a specific button inside a card or table row when multiple cards have identical buttons?

[Difficulty: Intermediate] What Interviewer Evaluates: Tests proficiency with locator filtering (.filter({ hasText })) and locator chaining.

Suggested Answer: You resolve this using locator filtering or chaining. With .filter({ hasText }), you first target the container (e.g. page.locator(".card").filter({ hasText: "Target Item" })) and then chain down to the button: card.getByRole("button", { name: "Add to Cart" }). This guarantees you interact with the exact card container regardless of page ordering.

Follow-Up Question: "How can you filter by another locator rather than plain text?"

Q5. What is the difference between expect(await locator.isVisible()).toBe(true) and await expect(locator).toBeVisible()?

[Difficulty: Intermediate] What Interviewer Evaluates: Crucial test of understanding web-first assertions vs generic synchronous assertions.

Suggested Answer: expect(await locator.isVisible()).toBe(true) checks visibility once immediately at that instant. If the element is fading in or waiting for an API response, it returns false and fails the test. In contrast, await expect(locator).toBeVisible() is a web-first assertion that repeatedly polls the DOM for up to 5 seconds until the element becomes visible, eliminating flakiness.

Follow-Up Question: "What is the default timeout for Playwright web-first assertions, and how can you customize it?"

Q6. Why are absolute XPath selectors (/html/body/div[2]/...) considered an anti-pattern in modern test automation?

[Difficulty: Beginner] What Interviewer Evaluates: Tests selector hygiene and ability to prevent maintenance nightmares.

Suggested Answer: Absolute XPath selectors depend on the exact nested structure of every parent and sibling tag between <html> and the target element. Any minor change by developers—such as adding a banner, wrapping inputs in a new grid div, or reordering columns—instantly breaks the XPath, causing high test maintenance overhead.

Follow-Up Question: "When is relative XPath acceptable in modern frameworks?"

Q7. How do you debug an element locator that is timing out in a Playwright test?

[Difficulty: Intermediate] What Interviewer Evaluates: Assesses practical troubleshooting methodology using Playwright Inspector and logging.

Suggested Answer: I run the test with npx playwright test --debug or insert await page.pause() before the failing step. This opens the Playwright Inspector and browser window. I use the Pick Locator tool or the Inspector search box to test selectors live against the DOM. I also check whether the element is inside an iframe, hidden by CSS, or waiting on a slow network request.

Follow-Up Question: "What CLI flag can you pass to launch tests with a visible browser without full debug mode?"

Q8. How does Playwright locate form inputs when using getByLabel vs getByPlaceholder?

[Difficulty: Beginner] What Interviewer Evaluates: Checks understanding of HTML form accessibility and locator mechanics.

Suggested Answer: getByLabel finds inputs associated with a <label> element (via the for/id attribute, aria-labelledby, or wrapping <label> tag). getByPlaceholder finds inputs by their HTML placeholder attribute. getByLabel is preferred for permanent form fields because labels are always visible, whereas placeholders disappear when the user begins typing.

Follow-Up Question: "What happens if a label has multiple matching form inputs?"

Q9. How should you prompt an AI coding assistant to generate reliable Playwright locators from HTML snippets?

[Difficulty: Intermediate] What Interviewer Evaluates: Evaluates AI fluency, prompt engineering discipline, and adherence to QA standards.

Suggested Answer: I provide the exact component HTML snippet, specify Playwright JavaScript (.js), and instruct the AI to prioritize user-facing locators (getByRole, getByLabel). I explicitly forbid brittle absolute XPath, positional indices, or dynamic CSS utility classes. Finally, I ask for a web-first assertion and verify the generated selector against the live DOM.

Follow-Up Question: "How do you detect if an AI-generated locator is fragile before adding it to your repository?"

Q10. How can you verify that a dynamic list or table contains exactly 5 items in Playwright?

[Difficulty: Beginner] What Interviewer Evaluates: Tests knowledge of web-first count assertions.

Suggested Answer: You use the web-first assertion await expect(locator).toHaveCount(5). For example: await expect(page.getByRole("listitem")).toHaveCount(5). This assertion automatically retries until the list renders all 5 items or times out, avoiding premature length assertions.

Follow-Up Question: "How would you iterate through all matching elements to extract their text values?"

1. What is the primary architectural difference between a Playwright Locator and a Selenium WebElement?

Key Concept: Playwright locators are lazy: they do not query the DOM until an action or assertion executes. They re-evaluate the query automatically and perform actionability checks, avoiding stale element exceptions.

2. Which of the following is considered Playwright's top-priority, gold standard locator method?

Key Concept: getByRole is the gold standard because it queries elements the way users and assistive technologies perceive them (by accessible role and accessible name), promoting resilient and accessible web applications.

3. What happens when you execute await page.getByRole("button").click() on a page containing 3 buttons?

Key Concept: Playwright enforces Strict Mode on actions. If a locator resolves to more than one element, Playwright immediately throws a strict mode violation error so you can disambiguate the target.

4. What is the recommended method to target a specific button inside a card without using fragile positional indices?

Key Concept: Using .filter({ hasText: "..." }) and locator chaining narrows the search strictly inside the matching card container, remaining resilient even if card ordering changes.

5. Why is await expect(locator).toBeVisible() preferred over expect(await locator.isVisible()).toBe(true)?

Key Concept: Web-first assertions like await expect(locator).toBeVisible() poll the DOM repeatedly for up to 5 seconds. In contrast, isVisible() evaluates instantaneously and fails if the element is mid-transition.

6. Which locator should be used for a text input that has a visible associated <label for="email">Email</label> tag?

Key Concept: page.getByLabel("Email") is designed specifically for form controls associated with a <label> element, verifying both the input and its accessibility relationship.

7. What command launches the Playwright Inspector alongside your browser for interactive locator debugging?

Key Concept: npx playwright test --debug opens the Playwright Inspector, pausing at each step and enabling the Pick Locator tool and live selector evaluation.

8. Why should you avoid using CSS classes like page.locator("button.bg-blue-600.px-4") in tests?

Key Concept: Styling classes (especially Tailwind utility classes) change often during UI redesigns. Tying automated tests to utility classes makes tests brittle and maintenance-heavy.

9. What does the web-first assertion await expect(cards).toHaveCount(4) do?

Key Concept: toHaveCount polls the DOM until the exact number of matching elements is present, allowing asynchronous lists to load without premature failure.

10. When an AI assistant suggests page.locator("/html/body/div[1]/form/div[2]/button"), what should you do?

Key Concept: Absolute XPath is an anti-pattern that breaks on any DOM modification. Senior engineers always refactor AI-suggested absolute XPaths into semantic, user-facing locators like getByRole.